

SIMATS ENGINEERING

Assessment Tool 3

Course Code: MLA03

Course Name: Reinforcement Learning for Environment and Agent

Course Outcome: CO5 — Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments (BL2, BL3, BL6)

Analytical Problem-Solving Assessment

Duration: 1 hr

Total Marks: 50

Faculty: Dr. G. A. Senthil

Code of Conduct:

I T.Ganesh(192425246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

T.Ganesh

Signature of the student:

Question 1

A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply Dynamic Programming to develop an optimal policy and explain how value iteration can improve navigation efficiency.

[Marks: 5]

1. Concept: Dynamic Programming in RL

Dynamic Programming (DP) methods solve an MDP when the full model (transition probabilities and reward function) is known. They compute optimal value functions and policies by repeatedly applying the Bellman equations, using bootstrapping (updating estimates from other estimates) instead of sampling.

For the warehouse robot, the grid layout, obstacle positions, and movement outcomes are known in advance, so DP is directly applicable — the aim is to compute, for every grid cell, the minimum-cost (shortest) way to reach the delivery point.

2. Problem Formulation

Element	Definition for this scenario
States (S)	Every free (non-obstacle) grid cell the robot can occupy
Actions (A)	Move Up / Down / Left / Right to an adjacent free cell
Transition $P(s' s,a)$	Deterministic: moving into a free cell succeeds; moving into an obstacle/wall keeps the robot in place
Reward $R(s,a,s')$	-1 per move (to encourage shortest path); large +reward on reaching the delivery goal

3. Value Iteration Algorithm

- Initialize $V(s) = 0$ for all states.

- Repeat for every state s : $V(s) \leftarrow \max \text{ over actions } a \text{ of } [R(s,a,s') + \gamma \cdot V(s')]$ (Bellman optimality backup).
- Continue sweeping over all states until the maximum change in $V(s)$ across an iteration falls below a small threshold θ (convergence).
- Extract the optimal policy $\pi^*(s) = \operatorname{argmax} \text{ over } a \text{ of } [R(s,a,s') + \gamma \cdot V(s')]$ using the converged value function.

4. How Value Iteration Improves Navigation Efficiency

Value iteration propagates cost-to-goal information backward from the delivery point across the grid, similar to a wavefront/Dijkstra-style expansion — cells closer to the goal converge to their correct value first, and this information ripples outward until every reachable cell has an accurate estimate of the shortest distance to the goal.

Because the greedy policy extracted from the converged value function always moves toward the neighbouring cell with the highest value (i.e., lowest remaining cost), the robot is guaranteed to follow the shortest obstacle-free path from any starting cell, and this policy is computed once offline and then reused instantly for any delivery request, making navigation highly efficient at run time.

5. Conclusion

By formulating the warehouse as an MDP and applying value iteration, the robot obtains a globally optimal, shortest-path policy for every cell in the grid; the backward propagation of value estimates is what allows the robot to efficiently avoid obstacles while minimizing travel distance, without needing to re-plan for each new delivery.

Question 2

A recommendation system collects customer interactions over multiple sessions. Apply Monte Carlo Control to update the policy and explain how the agent improves recommendations with experience.

[Marks: 5]

1. Concept: Monte Carlo Control

Monte Carlo (MC) methods learn purely from complete episodes of experience — no model of the environment is required, and no bootstrapping is used. Instead, the actual observed return (cumulative discounted reward) from a completed episode is used to update value estimates. This matches the recommendation scenario well, since a full user session (browsing, clicks, purchases, session end) is a natural episode boundary, and the true 'success' of a recommendation sequence is only known once the session concludes.

2. Problem Formulation

Element	Definition for this scenario
States (S)	User profile / recent browsing-interaction context at a point in the session
Actions (A)	Which item/content to recommend next
Episode	One complete user session, from first interaction to session end
Reward	+ for clicks/add-to-cart/purchase; 0 or small negative for ignored recommendations

3. Monte Carlo Control Algorithm

- Generate a full episode (session) by following the current policy π (e.g., epsilon-greedy over recommendation choices).

- For each state-action pair (s,a) that appears in the episode, compute the return $G_t = \text{sum of discounted rewards from time } t \text{ to the end of the session}$.
- Update the action-value estimate: $Q(s,a) \leftarrow \text{average of all observed returns } G_t \text{ following } (s,a) \text{ across episodes (first-visit or every-visit MC averaging)}$.
- Policy improvement: update the policy to be epsilon-greedy with respect to the updated $Q(s,a)$, i.e., mostly recommend $\text{argmax}_a Q(s,a)$ while retaining small exploration probability epsilon.
- Repeat over many customer sessions, continually refining Q and the policy (Generalized Policy Iteration).

4. How the Agent Improves with Experience

As more customer sessions are observed, the averaged returns for each (context, recommended-item) pair become statistically more reliable estimates of true long-term customer value, so $Q(s,a)$ estimates converge toward their true values and the epsilon-greedy policy increasingly favors recommendations that historically led to genuine engagement or purchase, rather than merely superficial clicks.

Because MC uses actual full-session outcomes rather than one-step bootstrapped estimates, it naturally captures delayed effects — e.g., a recommendation early in the session that leads to a purchase later — giving credit correctly to the actions that contributed to a successful session overall.

5. Conclusion

Monte Carlo Control lets the recommendation agent learn directly from complete, real customer sessions without needing an explicit model of user behaviour; as more sessions accumulate, the averaged-return Q -estimates become increasingly accurate, and the epsilon-greedy policy improvement step steadily biases recommendations toward higher long-term customer value.

Question 3

A delivery robot operates in an uncertain environment. Apply the SARSA algorithm to update the action-value function and explain how the learned policy adapts to environmental changes.

[Marks: 5]

1. Concept: SARSA (On-Policy TD Control)

SARSA (State-Action-Reward-State-Action) is an on-policy Temporal-Difference control algorithm: it updates the action-value function using the action the agent actually takes next (sampled from its current, exploring policy), rather than the theoretically best action. This makes SARSA well-suited to an uncertain, real-world delivery environment, since the learned values reflect the actual (exploration-inclusive) behaviour the robot will follow, making the resulting policy more cautious and realistic under uncertainty.

2. SARSA Update Rule

- Observe current state s and choose action a using the current policy (e.g., epsilon-greedy over Q).
- Execute a , observe reward r and next state s' .
- Choose the next action a' from state s' using the same current policy (this is the 'on-policy' step).
- Update: $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot Q(s',a') - Q(s,a)]$
- Set $s \leftarrow s'$, $a \leftarrow a'$, and repeat as the robot continues navigating.

3. How the Learned Policy Adapts to Environmental Changes

Because SARSA updates $Q(s,a)$ using the actually-executed next action a' (which includes occasional exploratory moves), it directly incorporates the consequences of the exploration the robot performs while encountering new obstacles, blocked routes, or changed traffic patterns — the value function is

continuously corrected using fresh, real transition data as the environment evolves.

If a previously good route becomes blocked or risky (e.g., a new obstacle appears), the temporal-difference error $r + \gamma Q(s',a') - Q(s,a)$ becomes strongly negative the next time that route is attempted, quickly reducing $Q(s,a)$ for that action and steering the epsilon-greedy policy toward alternative routes — allowing the robot to adapt its navigation policy online, without needing to restart training from scratch.

4. Conclusion

SARSA's on-policy, bootstrapped update lets the delivery robot's action-value function track the true consequences of the policy it currently follows (including exploration), so as the uncertain environment changes, TD errors quickly propagate the new information into $Q(s,a)$ and the resulting policy adapts its routes accordingly.

Question 4

A robotic arm performs continuous object manipulation. Apply the REINFORCE algorithm to develop a policy that maximizes task completion accuracy.

[Marks: 5]

1. Concept: REINFORCE (Monte Carlo Policy Gradient)

REINFORCE is a policy-gradient method that directly parameterizes and optimizes the policy $\pi_{\theta}(a|s)$ (e.g., a neural network with parameters θ outputting a distribution over continuous joint torques/positions), rather than learning value functions and deriving a policy indirectly. This makes it naturally suited to a robotic arm's continuous, high-dimensional action space, where discretizing actions (as required by Q-learning/SARSA) would be impractical.

2. REINFORCE Update Rule

- Run a full episode (one manipulation attempt) using the current policy π_{θ} , recording states, actions, and rewards.
- Compute the return $G_t = \text{sum of discounted rewards from time } t \text{ to the end of the episode}$, for every time step.
- Update the policy parameters via gradient ascent: $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t$, summed/averaged over the episode's time steps.
- (Optional but recommended) subtract a baseline $b(s_t)$ — e.g., an average return or a learned state-value estimate — from G_t to reduce variance: $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot (G_t - b(s_t))$.
- Repeat across many manipulation episodes, gradually shifting the policy toward action distributions that produced high returns.

3. How This Maximizes Task Completion Accuracy

The reward function is designed to reflect manipulation success (e.g., + reward for correctly grasping/placing the object, penalty for drops or

misalignment). Because $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$ increases the probability of actions that led to high returns and decreases the probability of those that led to low returns, repeated updates gradually reshape the policy's action distribution to favor motor commands that reliably complete the manipulation task.

As training progresses over many episodes, the variance-reduced gradient estimates (via the baseline) allow the continuous-control policy to converge more stably toward fine-grained joint movements that consistently maximize task-completion accuracy, rather than noisy, inconsistent motions.

4. Conclusion

REINFORCE's direct policy-gradient formulation is well matched to the robotic arm's continuous action space; by nudging policy parameters in the direction of log-probability gradients weighted by episode return (with a baseline for stability), the arm's policy is shaped over time to maximize successful, accurate task completion.

Question 5

An autonomous vehicle is trained using Q-Learning and Deep Q-Networks (DQN). Analyze the differences in learning performance, convergence speed, and scalability for complex driving environments.

[Marks: 5]

1. Overview

Q-Learning is a tabular, model-free, off-policy method that stores a separate Q-value for every discrete state-action pair. DQN replaces the table with a deep neural network function approximator $Q(s,a;\theta)$, enabling it to generalize over large or continuous state spaces (such as raw sensor/camera input) that tabular Q-Learning cannot handle.

2. Comparative Analysis

Dimension	Q-Learning (Tabular)	Deep Q-Network (DQN)
Learning performance	High accuracy but only for small, discretized state spaces; performance degrades sharply as state complexity grows	Learns effective policies directly from high-dimensional sensory input (images, LIDAR) by generalizing across similar states
Convergence speed	Fast and provably convergent for small state spaces, but requires visiting every state-action pair many times	Slower and less theoretically guaranteed (non-stationary targets, function-approximation instability), but converges practically with techniques like experience replay and target networks
Scalability	Does not scale — a realistic driving state space (continuous position, velocity,	Scales to complex, realistic driving environments by approximating Q-values, though requires significant computation

Dimension	Q-Learning (Tabular)	Deep Q-Network (DQN)
	sensor readings) is far too large to tabulate	(GPU training) and large amounts of data

3. Discussion

For a genuinely complex driving environment with continuous, high-dimensional sensor input, tabular Q-Learning is not practically viable — the state space explodes combinatorially, making both memory storage and adequate exploration of every state infeasible. DQN's neural-network generalization allows it to share learning across similar-looking driving situations it has never exactly seen before, which is essential for realistic autonomous driving.

The trade-off is that DQN sacrifices Q-Learning's simple convergence guarantees for scalability — it requires stabilization techniques (experience replay to break correlated samples, a separate target network to stabilize the bootstrapped target) to converge reliably in practice.

4. Conclusion

Q-Learning offers fast, guaranteed convergence but only for small, discretizable problems, whereas DQN trades some of that convergence stability for the generalization ability needed to scale to complex, high-dimensional, realistic driving environments — making DQN (with stabilization techniques) the practical choice for autonomous vehicles.

Question 6

A game-playing AI is implemented using DQN, Double DQN (DDQN), and Dueling DQN. Analyze the advantages of DDQN and Dueling DQN over the standard DQN architecture.

[Marks: 5]

1. Overview: The Problem with Standard DQN

Standard DQN selects and evaluates the next action using the same network's max operator: $\text{target} = r + \gamma \cdot \max_{a'} Q(s', a'; \theta)$. Because the max operator is applied to noisy, imperfect Q-estimates, this tends to systematically overestimate Q-values (a positive bias), which can lead to suboptimal policies, especially early in training when estimates are still inaccurate.

2. Double DQN (DDQN) — Advantage

- DDQN decouples action selection from action evaluation: the online network chooses the best next action ($\arg\max_{a'} Q(s', a'; \theta)$), but the target network evaluates that action's value ($Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$).
- This decoupling significantly reduces the systematic overestimation bias present in standard DQN, leading to more accurate value estimates and, in practice, more stable and often better-performing policies in game environments.

3. Dueling DQN — Advantage

- Dueling DQN splits the network into two streams after the shared feature layers: a state-value stream $V(s)$ and an action-advantage stream $A(s, a)$, recombined as $Q(s, a) = V(s) + (A(s, a) - \text{mean}_{a'} A(s, a))$.
- This architecture lets the network learn the value of being in a state independently of which specific action is taken — very useful in games where, in many states, the choice of action has little effect on the outcome (e.g., no immediate threat present), so the value stream can be

updated efficiently even without needing to evaluate every action's advantage separately.

- As a result, Dueling DQN typically learns faster and generalizes better across actions in states where action choice barely matters, improving overall sample efficiency.

4. Combined Advantage over Standard DQN

DDQN and Dueling DQN address two different weaknesses of standard DQN — value overestimation and inefficient value/advantage learning, respectively — and are commonly combined (Dueling-DDQN) to obtain both more accurate value estimates and faster, more robust learning than plain DQN, which is reflected in improved and more stable game-playing performance across many benchmark environments.

5. Conclusion

Compared to standard DQN, DDQN reduces harmful overestimation bias by decoupling action selection from evaluation, while Dueling DQN improves learning efficiency by separately modeling state value and action advantage — together making the game-playing agent's learning both more accurate and more sample-efficient than plain DQN.

Question 7

A smart drone navigation system uses A2C and A3C algorithms. Analyze the impact of parallel learning on convergence speed, computational efficiency, and policy stability.

[Marks: 5]

1. Overview

Both A2C (Advantage Actor-Critic) and A3C (Asynchronous Advantage Actor-Critic) combine a policy (actor) with a value function (critic) that estimates the advantage $A(s,a) = Q(s,a) - V(s)$ to reduce gradient variance. The key difference is parallelism: A2C runs multiple environment copies synchronously and applies one combined gradient update per step, while A3C runs multiple actor-learners asynchronously, each updating a shared global network independently.

2. Impact of Parallel Learning

Dimension	A2C (Synchronous)	A3C (Asynchronous)
Convergence speed	Good — batched, synchronized updates provide a stable, low-variance gradient signal each step	Often faster in wall-clock time — many workers explore different parts of the environment simultaneously, generating more diverse experience per unit time
Computational efficiency	Efficient on GPU (batched updates), but must wait for the slowest worker each step (synchronization overhead)	Efficient on multi-core CPUs — no waiting for synchronization, workers proceed independently, high hardware utilization
Policy stability	More stable — synchronized updates	Can be slightly less stable — asynchronous updates from

Dimension	A2C (Synchronous)	A3C (Asynchronous)
	avoid stale/conflicting gradients	different workers may use outdated ('stale') copies of the global network, introducing noise into training

3. Discussion for Drone Navigation

For a smart drone navigation system, A3C's parallel, asynchronous exploration across multiple simulated drones/environments allows the shared policy to experience a much wider variety of flight scenarios (varying obstacles, wind conditions) in the same wall-clock time, generally accelerating convergence toward a robust navigation policy.

However, this comes at the cost of some training-signal noise from stale gradients, which A2C avoids through synchronization at the expense of speed (idle time waiting on the slowest environment copy). In practice, A2C is often preferred when GPU-batched training and stable convergence are priorities, while A3C is favored when abundant CPU cores are available and faster wall-clock convergence is more important.

4. Conclusion

Parallelism speeds up learning in both algorithms by generating more diverse experience faster, but A3C's asynchronous nature trades some policy stability for greater wall-clock speed and CPU efficiency, whereas A2C's synchronous updates favor stability and GPU efficiency at the cost of synchronization overhead — the right choice depends on whether the drone system prioritizes raw training speed or stable, consistent convergence.

Question 8

A healthcare decision-support system operates with incomplete patient information. Analyze how a Partially Observable Markov Decision Process (POMDP) improves decision-making compared to a standard MDP.

[Marks: 5]

1. Overview: The Limitation of a Standard MDP Here

A standard MDP assumes the agent can directly observe the true, complete state of the environment at every step. In a healthcare setting, however, the patient's true underlying medical condition is never fully observable — the system only has access to noisy, incomplete signals (symptoms, lab results, vitals), so treating these observations as if they were the true state (as an MDP would) can lead to misinformed, potentially unsafe treatment decisions.

2. How a POMDP Improves Decision-Making

- A POMDP formally separates the true hidden state s (the patient's actual condition) from the observation o (test results, reported symptoms) generated via an observation function $O(o|s,a)$, acknowledging explicitly that the agent never sees s directly.
- The agent maintains a belief state — a probability distribution over possible true patient conditions — which is updated via Bayesian filtering each time a new observation (e.g., a new test result) arrives, rather than assuming any single observation is the ground truth.
- Decisions (e.g., which treatment or test to recommend next) are made based on the belief distribution, allowing the system to explicitly reason under uncertainty — for example, recommending an additional diagnostic test specifically to reduce uncertainty (information-gathering action) before committing to a risky treatment, something a standard MDP has no mechanism to represent.

- This naturally supports safer, more cautious decision-making: when belief uncertainty over the patient's condition is high, the POMDP-based policy can favor further information-gathering over premature, high-risk interventions.

3. Comparative Analysis

Aspect	Standard MDP	POMDP
State assumption	Full, direct observability of true state	True state hidden; only noisy observations available
Decision basis	Current observed state (assumed = true state)	Belief state (probability distribution over true states)
Handling of uncertainty	None — cannot represent diagnostic ambiguity	Explicitly modeled and updated via Bayesian belief filtering
Suitability for healthcare	Risks acting on mis-assumed certainty	Better matches the genuinely uncertain, incomplete-information nature of diagnosis

4. Conclusion

Because patient information is inherently incomplete and noisy, a POMDP's belief-state formulation lets the healthcare decision-support system reason honestly about its uncertainty and act (or gather more information) accordingly, resulting in more clinically appropriate and safer decisions than a standard MDP, which would incorrectly treat incomplete observations as ground truth.

Question 9

A robotic navigation system is trained using PPO, TRPO, and DDPG. Evaluate the suitability of each algorithm for continuous control tasks based on stability, sample efficiency, and computational complexity.

[Marks: 5]

1. Overview

All three algorithms are designed for continuous action spaces, making them suitable candidates for robotic navigation, but they differ substantially in how they update the policy and how efficiently/stably they learn.

2. Comparative Evaluation

Algorithm	Stability	Sample Efficiency	Computational Complexity
TRPO (Trust Region Policy Optimization)	Very high — enforces a strict KL-divergence trust-region constraint, preventing destructively large policy updates	Moderate — on-policy, so past data cannot be reused, but the safe updates avoid wasted, destabilizing steps	High — requires second-order optimization (conjugate gradient, Fisher-information matrix computation), making it computationally expensive per update
PPO (Proximal Policy Optimization)	High — approximates TRPO's trust-region behaviour using a simpler clipped surrogate objective,	Moderate — on-policy like TRPO (no replay buffer), but often needs fewer	Low–Moderate — only first-order (standard gradient descent) optimization, much cheaper

Algorithm	Stability	Sample Efficiency	Computational Complexity
	keeping updates within a safe range	samples in practice than naive policy gradient due to more efficient updates	than TRPO while retaining most of its stability benefits
DDPG (Deep Deterministic Policy Gradient)	Lower — off-policy actor-critic with a deterministic policy; prone to instability and sensitive to hyperparameters/noise settings	High — off-policy, uses a replay buffer so past experience can be reused many times, generally more sample-efficient than PPO/TRPO	Moderate — first-order optimization like PPO, but requires maintaining actor, critic, and target networks alongside a replay buffer

3. Suitability Discussion for Robotic Navigation

TRPO offers the strongest theoretical stability guarantees, making it attractive when safe, monotonic policy improvement is critical (e.g., a physical robot where a bad update could cause unsafe motion), but its computational overhead per update is a real practical cost.

PPO is generally the most practical default for robotic navigation: it retains much of TRPO's stability with far less computational cost, and is widely used in real robotic and simulation-to-real-world transfer settings due to this favorable trade-off.

DDPG's replay-buffer-based off-policy learning gives it the best raw sample efficiency of the three, which matters when real-robot interaction data is

expensive or slow to collect, but its instability and hyperparameter sensitivity mean it typically requires more careful tuning and monitoring during training than PPO.

4. Conclusion

TRPO is the most stable but computationally expensive, PPO offers the best practical balance of stability and computational cost (making it a strong general default), and DDPG offers the highest sample efficiency at the cost of training stability — so the best choice depends on whether the robotic navigation system prioritizes guaranteed stability (TRPO), a practical balance (PPO), or maximum data efficiency under real-hardware constraints (DDPG).

Question 10

A smart city traffic management system uses Multi-Agent Reinforcement Learning (MARL) to coordinate traffic signals. Evaluate the effectiveness of MARL in improving traffic flow while considering scalability, coordination, fairness, and ethical challenges.

[Marks: 5]

1. Overview

In this scenario, each intersection's traffic-signal controller can be modeled as an individual agent that observes local traffic conditions and selects signal phases, with the goal of collectively minimizing city-wide congestion. Because each intersection's decisions affect neighbouring intersections' traffic flow, this is fundamentally a multi-agent coordination problem rather than a set of independent single-agent problems.

2. Effectiveness Evaluation

Dimension	Evaluation
Scalability	Independent/decentralized MARL (each intersection learns its own policy) scales reasonably well to a city-wide network since agents don't need a centralized controller, but coordination quality tends to degrade as the number of interacting intersections grows unless some structured communication or shared information is used
Coordination	Fully independent agents (no communication) can produce locally optimal but globally poor outcomes (e.g., two agents each clearing their own queue but pushing congestion onto a shared connecting road); techniques such as shared reward signals, communication/attention mechanisms between neighbouring agents, or centralized-training-decentralized-execution improve coordination

Dimension	Evaluation
Fairness	A purely throughput-maximizing reward can systematically favor high-traffic arterial roads over low-traffic side streets/residential areas, leaving some routes with persistently long waits; fairness-aware reward terms (e.g., bounding maximum wait time per approach) are needed to avoid this bias
Ethical challenges	Must guarantee pedestrian safety and emergency-vehicle priority as hard constraints (not just reward terms); must avoid discriminatory outcomes across different neighbourhoods; and requires transparency/accountability for automated decisions that affect public infrastructure

3. Discussion

Overall, MARL is genuinely effective at improving traffic flow relative to static, independently-timed signals, because it can adapt to real-time, city-wide congestion patterns and — when agents are allowed some coordination — can reduce cascading congestion that independent (uncoordinated) signal control would create.

However, its real-world effectiveness is bounded by the quality of coordination mechanisms used (naive independent learning under-performs coordinated approaches), and responsible deployment requires embedding fairness constraints and hard safety rules (pedestrian crossing minimums, emergency-vehicle overrides) rather than relying purely on a learned, throughput-maximizing reward.

4. Conclusion

MARL can meaningfully improve city-wide traffic flow through adaptive, locally-responsive signal control, and its effectiveness scales reasonably well with decentralized learning, but achieving good coordination (rather than locally-optimal, globally-poor behaviour) and ensuring fairness and safety require deliberate design choices — shared/coordinated reward structures,

fairness constraints, and hard safety rules — rather than being guaranteed by MARL alone.